

COMP2271: Computer Graphics

Part 3: Lighting and Shading

Dr. Frederick Li

Department of Computer Science
Durham University

A 3D model's visual identity is determined by its **Material**, a high-level abstraction that acts as a container for the physical and mathematical rules governing how a surface interacts with light. Within this material, the **Albedo** represents the inherent **Colour**, or diffuse reflectivity, which serves as the base spectral data for every lighting calculation performed by the shader.

1. Definition

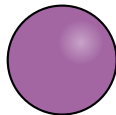
- In Three.js, colour is a property of the **Material**, not the Mesh geometry.
- The `THREE.Color` class manages Hex, RGB, and HSL conversions.

Implementation Snippet

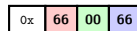
```
const myCol = new THREE.Color(0x660066);  
const mat = new THREE.MeshPhongMaterial({  
  color: myCol,  
  specular: 0x111111  
});
```

2. How it Works

- The material's **albedo** (base colour) acts as a multiplier for incoming light.
- Final Pixel = $uColor \times (Ambient + Diffuse + Specular)$.



Material Albedo (*uColor*)



Hex 66 = Dec 102

Red Green Blue

HSV mimics human-centric perception because it decouples the qualitative "tint" of a colour from its quantitative intensity and luminance, allowing for intuitive artistic control.

1. Definition

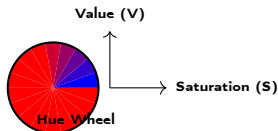
- **Hue (H):** The specific "tint" on a 0–360° colour wheel.
- **Saturation (S):** The vibrancy or "purity" of the colour.
- **Value/Lightness (V/L):** Total brightness/luminance.

2. How it Works (Advantages)

- **Procedural Jittering:** Vary 'Hue' while keeping 'S' and 'V' constant for object variety.
- **Intuitive Shading:** Reducing 'Value' creates natural shadows without shifting the hue.

Three.js Implementation

```
const col = new THREE.Color();  
col.setHSL(0.1, 0.8, 0.5); // (H, S, L)
```



The World Without Light

A Thought Experiment

Imagine you are standing in a perfectly dark room.
You hold a perfect sphere in your hand.

- You know its geometry is flawless. You know its position in space (x, y, z) .

The World Without Light

A Thought Experiment

Imagine you are standing in a perfectly dark room.
You hold a perfect sphere in your hand.

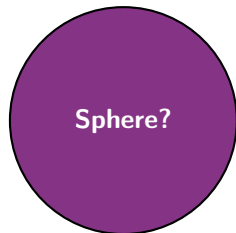
- You know its geometry is flawless. You know its position in space (x, y, z) .
- But if you turn on a light that emits **no shadows** and **no highlights**, what do you see?

The World Without Light

A Thought Experiment

Imagine you are standing in a perfectly dark room.
You hold a perfect sphere in your hand.

- You know its geometry is flawless. You know its position in space (x, y, z) .
- But if you turn on a light that emits **no shadows** and **no highlights**, what do you see?
- You see a **flat circle**.



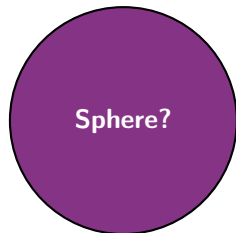
No depth cues

The World Without Light

A Thought Experiment

Imagine you are standing in a perfectly dark room.
You hold a perfect sphere in your hand.

- You know its geometry is flawless. You know its position in space (x, y, z) .
- But if you turn on a light that emits **no shadows** and **no highlights**, what do you see?
- You see a **flat circle**.



No depth cues

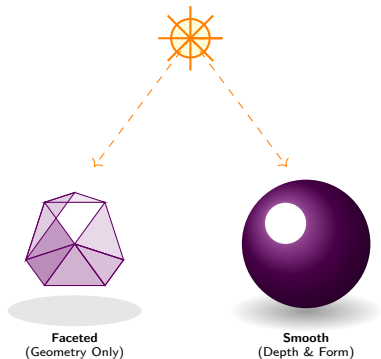
The Insight

Geometry provides the *skeleton* of a 3D object, but Light provides the *flesh*.
Without the math of lighting, 3D graphics is just 2D geometry.

- So far, we can draw 3D objects and project them onto our 2D screen.

- So far, we can draw 3D objects and project them onto our 2D screen.
- However, if we render them with a single, flat colour, they look artificial and unconvincing.

- So far, we can draw 3D objects and project them onto our 2D screen.
- However, if we render them with a single, flat colour, they look artificial and unconvincing.
- They lack a sense of depth, shape, and substance.



Defining Lighting in Computer Graphics

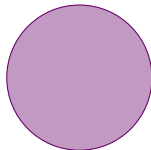
From Flat Shapes to 3D Substance

What is Lighting in CG?

- It is the mathematical simulation of how light interacts with object (mesh) surfaces.
- We use algorithms to calculate the colour and brightness of pixels based on virtual light sources.

Why is it Essential?

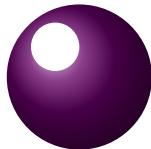
- **Reveals Form:** Without lighting, a 3D sphere is indistinguishable from a flat 2D circle.
- **Conveys Material:** It tells our brains what an object is made of (e.g., shiny plastic, dull clay, or smooth metal) by showing specular highlights and diffuse scattering.
- **Creates Depth:** Shadows and gradients provide the crucial depth cues needed for the illusion of three-dimensionality.



Geometry (2D Silhouette)



+ Lighting Math



Reality (3D Volume & Material)

Global vs. Local Illumination

Approximations in Rasterisation

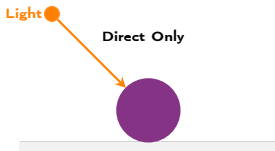
- **Global Illumination (GI):** Simulates light bouncing infinitely between surfaces (indirect lighting, colour bleeding). Physically accurate but *extremely computationally expensive*.
- **Local Illumination:** Calculates light for an object considering *only* the direct rays from a light source. It ignores how objects reflect light onto one another.

Rasterisation & Illumination

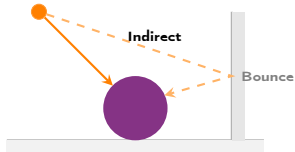
Standard **rasterisation-based rendering** (like OpenGL) fundamentally relies on **Local Illumination**.

Why? Rasterisation processes the scene object-by-object, projecting polygons to the screen independently. It lacks the scene-wide spatial awareness required to trace bouncing light rays between different objects efficiently. We instead use mathematical "hacks" (like ambient lighting) to fake global bounces!

Local Illumination



Global Illumination



Lighting vs. Shading Models

Physics vs. Strategy

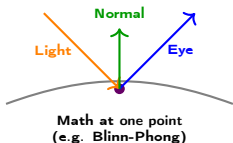
These terms are often used interchangeably, but they represent two distinct steps in the pipeline:

1. Lighting Model (The Formula)

A mathematical approximation of photon interaction at a single point.

Core Concern: Light transport physics (Reflection, Refraction, Absorption).

Question: *"What colour should the pixel be based on vectors?"*

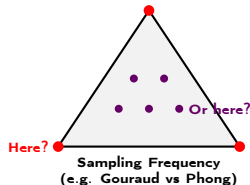


2. Shading Model (The Frequency)

The algorithm used to distribute that lighting formula across a 3D surface.

Core Concern: Performance vs. Quality (Where do we sample the math?).

Question: *"Where and how often do we run the formula?"*



Normal Vectors for Lighting Equation

Mathematics, Implementation, and Shading Role

1. Mathematical Approaches

- **Per-Face (Flat):** A single normal represents the entire polygon. It is calculated using the **Cross Product** of two edges:

$$\vec{N} = \frac{(\mathbf{V}_1 - \mathbf{V}_0) \times (\mathbf{V}_2 - \mathbf{V}_0)}{\|(\mathbf{V}_1 - \mathbf{V}_0) \times (\mathbf{V}_2 - \mathbf{V}_0)\|}$$

- **Per-Vertex (Smooth):** Each vertex has a unique normal. It is calculated by **averaging** the normals of all faces sharing that vertex:

$$\vec{N}_v = \frac{\sum \vec{n}_{face}}{\|\sum \vec{n}_{face}\|}$$

2. Three.js Command

```
// Automatically calculates/updates  
// vertex normals for a geometry  
geometry.computeVertexNormals();
```

3. Role in Shading

- **Diffuse:** brightness is a direct result of the **Dot Product** between the normal ($\vec{N}$) and light ($\vec{L}$).
- **Specular:** The highlight reaches maximum intensity when the normal ($\vec{N}$) aligns with the Halfway vector ($\vec{H}$).
- **Illusion:** Normals allow low-poly meshes to simulate smooth, curved surfaces.

Important Note on Scaling

To maintain correct lighting after non-uniform scaling, always multiply normals by the Normal Matrix ($\mathbf{M}^{-1})^T$.

We will focus on the classic **Blinn-Phong lighting model**.

- It is an **empirical model**: it approximates the look of lighting rather than simulating the complex physics perfectly.

A Classic and Efficient Approach

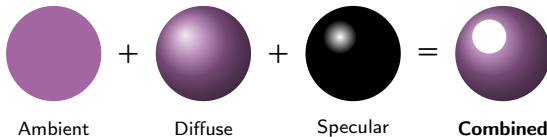
We will focus on the classic **Blinn-Phong lighting model**.

- It is an **empirical model**: it approximates the look of lighting rather than simulating the complex physics perfectly.
- **Why is it popular?** It is computationally efficient (fast!) and produces good-looking results that are easy to control.

We will focus on the classic **Blinn-Phong lighting model**.

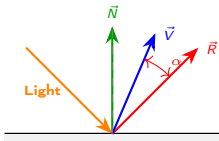
- It is an **empirical model**: it approximates the look of lighting rather than simulating the complex physics perfectly.
- **Why is it popular?** It is computationally efficient (fast!) and produces good-looking results that are easy to control.
- It simplifies lighting by breaking it down into three distinct components, which are then added together.

$$\text{Final Colour} = \text{Ambient} + \text{Diffuse} + \text{Specular}$$



Conceptual Differences & Trade-offs

1. Phong Model (Strict Reflection)



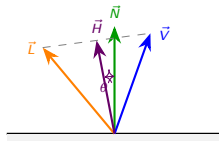
The Mechanics

Concept: "Is the eye aligned with the reflection?" Specular is brightest when View ($\vec{V}$) aligns with Reflection ($\vec{R}$).

Math (Expensive): We must calculate the exact reflection vector $\vec{R}$ per pixel:

$$\vec{R} = 2(\vec{N} \cdot \vec{L})\vec{N} - \vec{L}$$

2. Blinn-Phong Model (Halfway Approximation)



The Mechanics

Concept: "Is the surface angled between light and eye?" Specular is brightest when Normal ($\vec{N}$) aligns with Halfway ($\vec{H}$).

Math (Efficient): We calculate the Halfway vector $\vec{H}$ using simple addition:

$$\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|}$$

Summary

Phong is physically intuitive but computationally expensive to calculate $\vec{R}$. **Blinn-Phong** is an optimised approximation that is faster and often produces more realistic (less "plastic") results at steep angles.

The Global Glow

Approximating Global Illumination (Ambient Lighting)

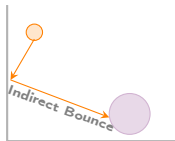
The Reality: Global Illumination

- In the physical world, light bounces infinitely between surfaces, a phenomenon called *global illumination*.
- This constant scattering ensures that even areas hidden from direct light are never truly pitch black.
- However, simulating trillions of these physical bounces is *extremely expensive* for real-time GPUs.

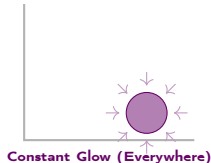
The Graphics "Hack"

- We use the **ambient component** as a highly efficient "hack" to simulate this global glow.
- It applies a constant, low-intensity light to every point on every object equally, regardless of orientation.

Physical Reality



Ambient Approximation



The Result

Shadows are prevented from falling into pure blackness, ensuring objects retain their 3D shape and presence even without direct light.

The calculation is a simple multiplication:

$$\text{Ambient Colour} = \text{Strength} \times \text{Light Colour} \times \text{Object Colour}$$

```
// Fragment Shader
uniform vec3 u_lightColor;
uniform vec3 u_objectColor;

void main() {
    float ambientStrength = 0.1; // A small constant factor
    vec3 ambient = ambientStrength * u_lightColor;

    vec3 result = ambient * u_objectColor;
    gl_FragColor = vec4(result, 1.0);
}
```

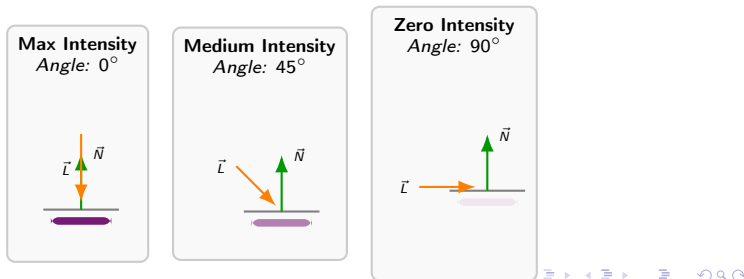
Result: It provides a base colour, but on its own, it looks flat and uninteresting. Its value comes from being combined with other components.

Revealing an Object's Form

Lambert's Cosine Law

Diffuse reflection occurs when light hits a matte surface, scattering in all directions.

- **Perceiving Depth:** This is the most critical component for revealing an object's 3D shape and substance.
- **Angle Dependency:** Brightness depends entirely on the angle between the light source and the surface normal.
- **Lambert's Law:** Surface intensity is at its maximum when light hits perpendicularly (0°) and reaches zero at grazing angles (90°).



Geometric Logic & Vector Transition

From Light Spreading to Dot Products

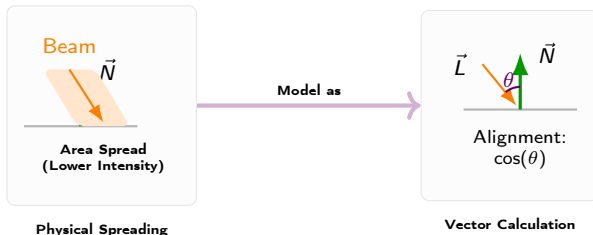
The Geometry of Intensity

- **Energy Density:** At 0° , a "bundle" of light hits a small area. At 60° , that same energy is spread over **twice** the area.
- **Result:** The intensity per pixel drops as the angle increases.

Extension: The Clamping Rule

What happens if light comes from **behind** the surface?

- $\cos(\theta)$ becomes negative.
- We use $\max(0, \vec{N} \cdot \vec{L})$ to "clamp" the light to zero, preventing negative colours.



Takeaway: We don't need to measure "area." We only need the **Dot Product** of the unit vectors $\vec{N}$ and $\vec{L}$.

The dot product of two *unit* (normalised) vectors gives the cosine of the angle between them.

$$\text{Diffuse Colour} = \max(0, \vec{N} \cdot \vec{L}) \times \text{Light Colour}$$

- $\vec{N}$ is the surface normal vector.
- $\vec{L}$ is the direction vector from the surface point to the light.
- The $\max(0, \dots)$ is crucial to prevent surfaces facing away from light (negative dot product) from having negative colour.

```
// Fragment Shader
varying vec3 v_normal;
uniform vec3 u_lightDirection;

// Normalize vectors!
vec3 normal = normalize(v_normal);
vec3 lightDir = normalize(u_lightDirection);

// Calculate diffuse intensity
float diff = max(dot(normal, lightDir), 0.0);
vec3 diffuse = diff * u_lightColor;
```

The Micro-surface Intuition

Alignment and the Shininess Exponent

The Mirror-Like Reflection

- Specular reflection simulates the "glint" seen on polished surfaces like metal or plastic.
- **Geometric Alignment:** The highlight is at its peak when the viewer ($\vec{V}$) is perfectly aligned with the reflected light ray ($\vec{R}$).

Extension: Surface Roughness

In reality, "rougher" surfaces have microscopic bumps that scatter the specular light.

- **Low Exponent:** Simulates a rougher surface; the highlight "spreads" out.
- **High Exponent:** Simulates a perfectly smooth surface; the light stays concentrated.

Rough Surface
Exponent: 2–8



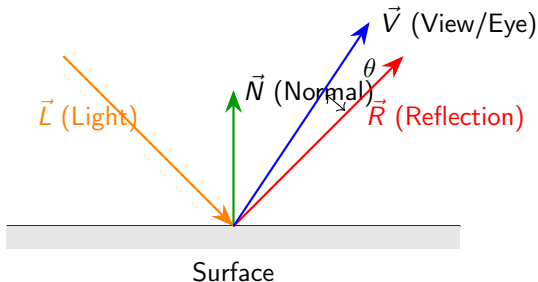
Polished
Exponent: 32–64



Highly Glossy
Exponent: 128+



Physics Hint: A higher exponent (n) creates a smaller, sharper highlight, simulating a *microscopically smoother surface*.



The highlight is strongest when the angle θ between $\vec{V}$ and $\vec{R}$ is zero.

Calculating the reflection vector $\vec{R}$ can be a bit slow. James Blinn introduced a clever optimisation: the **halfway vector** ($\vec{H}$).

- The halfway vector $\vec{H}$ is a unit vector that lies exactly halfway between the light vector $\vec{L}$ and the view vector $\vec{V}$.

Calculating the reflection vector $\vec{R}$ can be a bit slow. James Blinn introduced a clever optimisation: the **halfway vector** ($\vec{H}$).

- The halfway vector $\vec{H}$ is a unit vector that lies exactly halfway between the light vector $\vec{L}$ and the view vector $\vec{V}$.

$$\vec{H} = \text{normalise}(\vec{L} + \vec{V})$$

Calculating the reflection vector $\vec{R}$ can be a bit slow. James Blinn introduced a clever optimisation: the **halfway vector** ($\vec{H}$).

- The halfway vector $\vec{H}$ is a unit vector that lies exactly halfway between the light vector $\vec{L}$ and the view vector $\vec{V}$.



$$\vec{H} = \text{normalise}(\vec{L} + \vec{V})$$

- **New Principle:** The highlight is at its maximum when this halfway vector $\vec{H}$ aligns with the surface normal $\vec{N}$.

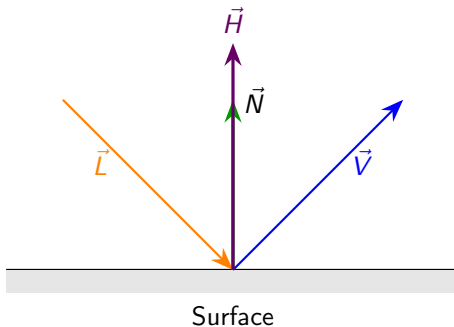
Calculating the reflection vector $\vec{R}$ can be a bit slow. James Blinn introduced a clever optimisation: the **halfway vector** ($\vec{H}$).

- The halfway vector $\vec{H}$ is a unit vector that lies exactly halfway between the light vector $\vec{L}$ and the view vector $\vec{V}$.

-

$$\vec{H} = \text{normalise}(\vec{L} + \vec{V})$$

- **New Principle:** The highlight is at its maximum when this halfway vector $\vec{H}$ aligns with the surface normal $\vec{N}$.
- This is computationally cheaper and often gives softer, more pleasing highlights (vs. Traditional Phong Model). This is the standard implementation.



We now check the angle between $\vec{N}$ and $\vec{H}$ instead of $\vec{V}$ and $\vec{R}$.

$$\text{Specular} = (\max(0, \vec{N} \cdot \vec{H}))^{\text{shininess}} \times \text{Strength}$$

```
// Fragment Shader
// ... requires camera position (view) and fragment position
vec3 lightDir = normalize(u_lightPosition - v_fragPosition);
vec3 viewDir = normalize(u_viewPosition - v_fragPosition);
vec3 halfwayDir = normalize(lightDir + viewDir);

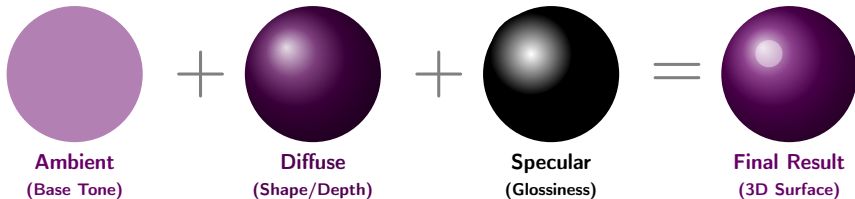
// Specular calculation
float shininess = 32.0; // Higher value = sharper highlight
float spec = pow(max(dot(normal, halfwayDir), 0.0), shininess);

// Final specular color
float specularStrength = 0.5;
vec3 specular = specularStrength * spec * u_lightColor;
```

Putting It All Together

The Additive Lighting Model

The power of the Blinn-Phong model lies in its **additive nature**. We calculate three distinct "layers" of light and stack them on top of each other.



The Final Formula

$$I_{total} = (k_a \cdot I_a) + (k_d \cdot I_d \cdot \max(N \cdot L, 0)) + (k_s \cdot I_s \cdot \max(N \cdot H, 0)^\alpha)$$


```
// Fragment Shader (Full Blinn-Phong)
// ... all uniforms (lightColor, objectColor, lightPos, viewPos)
// ... all varyings (v_normal, v_fragPosition)

void main() {
    // Common vectors
    vec3 normal = normalize(v_normal);
    vec3 lightDir = normalize(u_lightPosition - v_fragPosition);
    vec3 viewDir = normalize(u_viewPosition - v_fragPosition);
    vec3 halfwayDir = normalize(lightDir + viewDir);

    // 1. Ambient
    vec3 ambient = 0.1 * u_lightColor;

    // 2. Diffuse
    float diff = max(dot(normal, lightDir), 0.0);
    vec3 diffuse = diff * u_lightColor;

    // 3. Specular
    float spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0);
    vec3 specular = 0.5 * spec * u_lightColor;

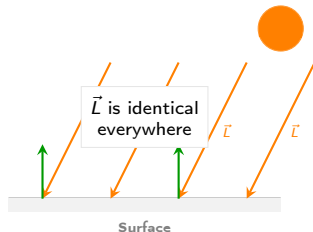
    // Combine and multiply by object color
    vec3 result = (ambient + diffuse + specular) * u_objectColor;
    gl_FragColor = vec4(result, 1.0);
}
```

Simulating the Sun (Directional Lighting)

Infinite Distance, Parallel Rays

The Concept

- Represents a light source that is **infinitely far away** (e.g., the Sun).
- Because the distance is effectively infinite, the light rays arrive **parallel** to each other.
- **Key Property:** The Light Vector ($\vec{L}$) is **constant**. It is exactly the same for every single vertex and pixel in the scene.
- No position, only direction.



The Directional Light Pipeline

Vertex Preparation vs. Fragment Calculation

1. Vertex Shader (Preparation)

Responsibility: Transform geometry and pass data.

- Transforms positions to Clip Space.
- Transforms normals to World Space so they match the light direction.

```
// Attributes & Uniforms
in vec3 a_position;
in vec3 a_normal;
uniform mat4 u_mvMatrix;
uniform mat4 u_normalMatrix; // (M^-1)^T

// Output to Fragment Shader
out vec3 v_normal;

void main() {
    // 1. Position -> Clip Space
    gl_Position = u_mvMatrix * vec4(a_position, 1.0);
    ;

    // 2. Normal -> World Space
    // Pass to Fragment Shader for interpolation
    v_normal = mat3(u_normalMatrix) * a_normal;
}
```

2. Fragment Shader (Calculation)

Responsibility: Calculate pixel brightness.

- Key Optimisation:** Because the light is infinitely far away, all rays are parallel. This means $\vec{L}$ is **identical** for every pixel, so we skip the expensive step of calculating a vector from the fragment to the light source.

```
// Inputs
in vec3 v_normal;
uniform vec3 u_lightDirection; // Fixed!
uniform vec3 u_lightColor;

void main() {
    // 1. Re-normalize the interpolated normal
    vec3 N = normalize(v_normal);

    // 2. Get Light Vector (Negate direction)
    // Directional Light L is the SAME for every pixel
    vec3 L = normalize(-u_lightDirection);

    // 3. Diffuse Calculation (Lambert)
    float diff = max(dot(N, L), 0.0);

    vec3 result = diff * u_lightColor;
    gl_FragColor = vec4(result, 1.0);
}
```

Performance Note

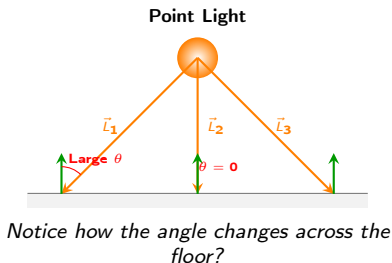
Directional lights are the cheapest to calculate because $\vec{L}$ is constant. Unlike Point Lights, we skip the expensive distance calculation ($P_{light} - P_{frag}$) and attenuation logic.

Simulating Bulbs and Fire (Point Lighting)

Local Position, Radial Spread, Attenuation

The Concept

- Represents a light with a specific **3D position** in the world.
- Light rays emit radially in all directions.
- **Key Property 1:** The Light Vector ($\vec{L}$) **changes** for every pixel ($\vec{L} = P_{light} - P_{frag}$).
- **Key Property 2 (Attenuation):** Light gets weaker as it travels further.



Point Light Pipeline & Attenuation

The Inverse Square Law ($I \propto 1/d^2$)

1. Vertex Shader (The Setup)

Responsibility: Calculate World Position. We need the actual 3D position of the surface, not its screen position.

```
out vec3 v_fragPos; // Pass to Fragment

void main() {
    // Transform Local -> World
    vec4 worldPos = u_model * vec4(a_pos, 1.0);
    v_fragPos = vec3(worldPos);

    gl_Position = u_mvp * vec4(a_pos, 1.0);
}
```

2. Fragment Shader (The Math)

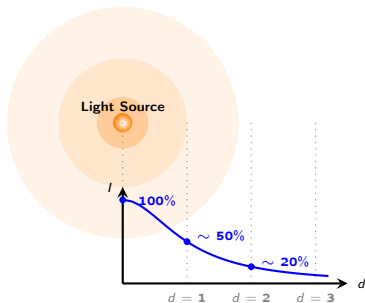
Responsibility: Calculate Distance & Falloff.

```
uniform vec3 u_lightPos;

void main() {
    // 1. Get Distance (d)
    float d = length(u_lightPos - v_fragPos);

    // 2. Calculate Attenuation (Linear/Quadratic)
    // Formula: 1 / (Kc + Kl*d + Kq*d^2)
    float att = 1.0 / (1.0 + 0.09*d + 0.032*d*d);

    // 3. Apply to Light
    vec3 finalLight = baseColor * att;
}
```



Tunable Falloff

We rarely use pure $1/d^2$. By tuning the Linear (K_l) and Quadratic (K_q) terms in the denominator, artists can control exactly how far the light reaches.

The Key to Correct Lighting

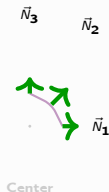
Geometric Data vs. Transformation Distortion

- **Surface Orientation:** Lighting depends on the **Normal** ($\vec{N}$), which must stay perpendicular to the surface to calculate light reflection correctly.
- **The Smoothness Illusion:** Curved objects use unique normals at each vertex (pointing away from the center) to blend light and hide flat triangle edges.

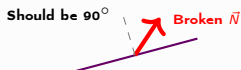
The Transformation Trap

Scaling an object uniformly is fine, but **non-uniform scaling** (e.g., stretching only the X-axis) distorts the normal's direction, making it no longer perpendicular to the surface.

1. Normals on Curves

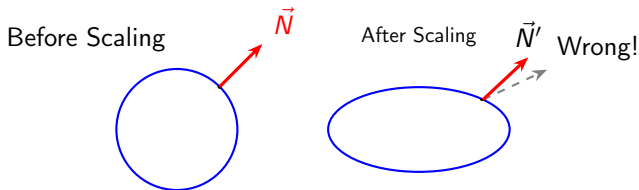


2. Scaling Distortion



The Normal Matrix

Simply multiplying the normal by the model matrix is **incorrect**, especially with non-uniform scaling.



A non-uniform scale distorts the object. The naively transformed normal (dashed gray) is no longer perpendicular to the surface.

The Correct Method

To fix this, we must multiply the normal vector by the **inverse transpose** of the model matrix.

$$\vec{N}_{\text{transformed}} = (\mathbf{M}^{-1})^T \vec{N}_{\text{original}}$$

This "Normal Matrix" is calculated on the CPU and passed to the vertex shader.

Where Do We Calculate the Light?

The Strategy of Sampling

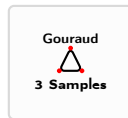
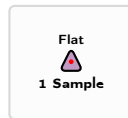
We have the **Lighting Model** (the math). Now we need a **Shading Model** (the plan) to apply that math to our geometry.

The Core Decision

How often do we sample the lighting equation across a surface?

The Performance-Quality Trade-off:

- Lighting math is "expensive" for the GPU.
- **Low Frequency:** Calculate once per face (Fast, but blocky).
- **High Frequency:** Calculate once per pixel (Slower, but beautiful).



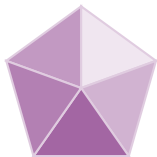
The choice of shading model determines if an object looks like **folded paper** or **polished glass**.

The 1975 Revolution

From "Cut Paper" to "Plastic"

1. Before 1975 (Flat Shading)

"The World of Polygons"



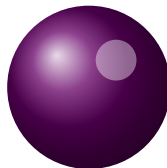
Faceted Geometry

The Reality

Objects looked like folded paper. Every triangle was a single, flat colour. No smoothness existed.

2. The Revolution (Phong)

"The Birth of Shine"



Interpolated Normals

The Idea

Bui Tuong Phong's Insight: *"Don't change the geometry. Fake the curve by interpolating the light across the surface."*

This simple trick creates the illusion of smooth surfaces on jagged meshes.

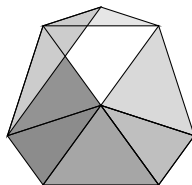
- **Method:** The lighting calculation is performed only **once for an entire triangle**.

- **Method:** The lighting calculation is performed only **once for an entire triangle**.
- **Result:** Every pixel in the triangle is assigned the exact same colour.

- **Method:** The lighting calculation is performed only **once for an entire triangle**.
- **Result:** Every pixel in the triangle is assigned the exact same colour.
- **Appearance:** Produces a faceted, "low-poly" look where each triangle is clearly visible. It cannot create smooth curved surfaces.

- **Method:** The lighting calculation is performed only **once for an entire triangle**.
- **Result:** Every pixel in the triangle is assigned the exact same colour.
- **Appearance:** Produces a faceted, "low-poly" look where each triangle is clearly visible. It cannot create smooth curved surfaces.
- **Use Cases:** Good for stylised art or debugging.

Flat Shading

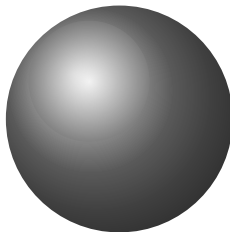


- **Method:** The full Blinn-Phong lighting calculation is performed at **each vertex** of a triangle.

- **Method:** The full Blinn-Phong lighting calculation is performed at **each vertex** of a triangle.
- **Process:**
 - 1 Vertex Shader computes a colour for each vertex.
 - 2 These vertex **colours** are then smoothly interpolated across the face of the triangle by the rasteriser.

- **Method:** The full Blinn-Phong lighting calculation is performed at **each vertex** of a triangle.
- **Process:**
 - 1 Vertex Shader computes a colour for each vertex.
 - 2 These vertex **colours** are then smoothly interpolated across the face of the triangle by the rasteriser.
- **Appearance:** Produces surfaces that appear smooth. A big improvement!

Gouraud Shading



Limitation

Specular highlights can be problematic. If a highlight should appear in the middle of a large triangle, but isn't near a vertex, it will be missed entirely.

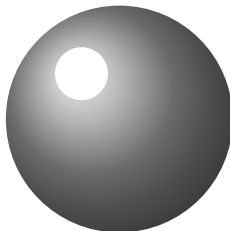
- **Method:** The lighting calculation is performed for **every single fragment (pixel)**.

- **Method:** The lighting calculation is performed for **every single fragment (pixel)**.

- **Process:**

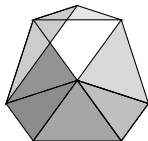
- 1 Vertex Shader passes the vertex **normals** to the rasteriser.
- 2 These normals are smoothly interpolated across the face of the triangle.
- 3 Fragment Shader receives a high-precision, interpolated normal for each pixel and performs the full lighting calculation there.

Phong Shading



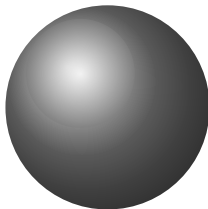
This is the modern standard. It is more expensive, but GPUs are highly optimised for this exact task, producing beautifully smooth surfaces and crisp, accurate highlights.

Flat



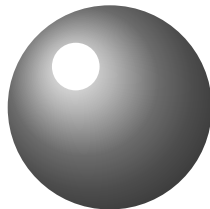
Lighting per-triangle.
Faceted look.

Gouraud (Smooth)



Lighting per-vertex. Smooth,
but inaccurate highlights.

Phong



Lighting per-fragment.
Smooth and accurate.

Figure: A low-polygon sphere rendered with the three primary shading models.

The Performance-Fidelity Trade-off

Choosing the Right Sampling Frequency

Selecting a shading model is a balance between the **"Physics Bill"** (computational cost) and the **Visual Illusion**.

Attribute	Flat Shading	Gouraud	Phong Shading
Calculation	Per-Triangle	Per-Vertex	Per-Fragment
Visual Quality	Faceted	Smooth	Ultra-Smooth
GPU Cost	Very Low	Low	High
Highlights	None	Inaccurate	Pixel-Perfect
Standard	Stylised/Low-Poly	Largely Obsolete	Modern Default



FLAT



GOURAUD



PHONG

- **Takeaway:** Use **Flat** for artistic "low-poly" styles , but **Phong** for everything else to ensure highlights behave correctly across surfaces.

While understanding GLSL is crucial, modern libraries like Three.js provide high-level abstractions. You don't write the lighting shaders yourself.

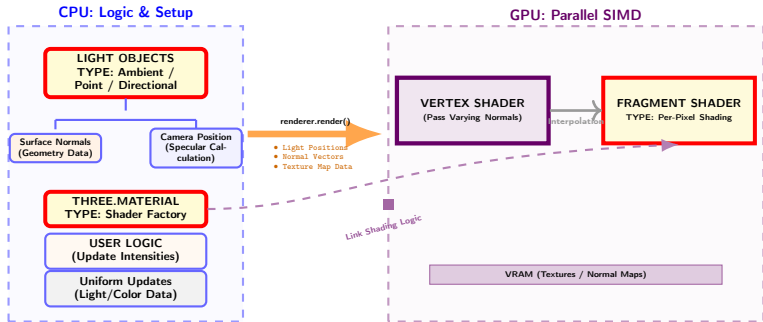
- Instead, you select a pre-built **Material** (e.g., 'MeshPhongMaterial').
- You add **Light** objects to your scene.
- The library handles creating the shaders and sending the uniforms for you.

```
// 1. Use a material that reacts to light
const material = new THREE.MeshPhongMaterial({
  color: 0xddddddd,    // Object's base color
  specular: 0x111111,  // Specular highlight color
  shininess: 30        // Shininess factor
});
const sphere = new THREE.Mesh(sphereGeometry, material);
scene.add(sphere);

// 2. Add light sources to the scene
const light = new THREE.PointLight(0xffffff, 1, 100);
light.position.set(10, 10, 10);
scene.add(light);
const ambientLight = new THREE.AmbientLight(0x404040);
scene.add(ambientLight);
```

Lighting & Shading in Three.js

Identifying Lighting Components & Shading Responsibilities



Lighting & Shading Responsibilities

- **Light Objects:** CPU defines light positions and types (Point, Directional) to be sent as uniforms.
- **Three.Material:** Acts as a "Shader Factory," stitching together GLSL code for specific lighting models (e.g., Phong, Lambert).
- **Fragment Shader:** The "Worker" stage where the final color is calculated per pixel using light vectors and surface normals.
- **Interpolation:** The GPU automatically smooths vertex normals across the triangle surface for the Fragment Shader.

- **Lighting** is what gives 3D objects the illusion of shape, depth, and material.

- **Lighting** is what gives 3D objects the illusion of shape, depth, and material.
- The **Blinn-Phong Lighting Model** is an efficient way to approximate lighting with three components:
 - **Ambient:** Base global light, prevents pure black shadows.
 - **Diffuse:** Scatters light based on the surface normal, revealing shape.
 - **Specular:** Creates shiny highlights based on view and light direction.

- **Lighting** is what gives 3D objects the illusion of shape, depth, and material.
- The **Blinn-Phong Lighting Model** is an efficient way to approximate lighting with three components:
 - **Ambient:** Base global light, prevents pure black shadows.
 - **Diffuse:** Scatters light based on the surface normal, revealing shape.
 - **Specular:** Creates shiny highlights based on view and light direction.
- **Phong Shading** (per-fragment) is the modern standard, calculating lighting for every pixel to produce high-quality, accurate results.

- **Lighting** is what gives 3D objects the illusion of shape, depth, and material.
- The **Blinn-Phong Lighting Model** is an efficient way to approximate lighting with three components:
 - **Ambient:** Base global light, prevents pure black shadows.
 - **Diffuse:** Scatters light based on the surface normal, revealing shape.
 - **Specular:** Creates shiny highlights based on view and light direction.
- **Phong Shading** (per-fragment) is the modern standard, calculating lighting for every pixel to produce high-quality, accurate results.
- High-level libraries like **Three.js** abstract these complex details away, allowing developers to focus on scene creation.

Summary: The 3 Layers of 3D Realism

1. Shape (The Skeleton)

Geometry defines *where* it is.

It is just a collection of **vertices** and flat **facets**.

2. Substance (The Flesh)

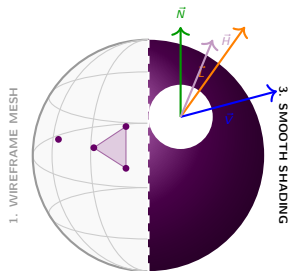
Blinn-Phong Model defines *what* it is.

- **Ambient:** Global presence.
- **Diffuse:** Matte form ($\vec{N} \cdot \vec{L}$).
- **Specular:** Shiny glint ($\vec{N} \cdot \vec{H}$).

3. Smoothness (The Skin)

Phong Shading defines *how* it feels.

- Calculate per-pixel, not per-vertex.



CRITICAL TRAP:
Non-Uniform Scale?
Fix Normals: $\vec{N}' = (\mathbf{M}^{-1})^T \vec{N}$

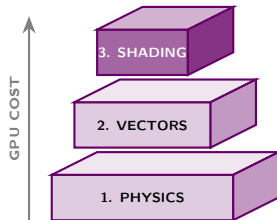
How to Approach This Lecture

A Layered Learning Strategy

To master these concepts, treat the lecture as three distinct layers of a 3D "Reality Recipe."

1 Layer 1: The Physics (The "Why")

- Understand the **Blinn-Phong Components**: Ambient (Global), Diffuse (Matte), and Specular (Shiny).
- *Study Goal*: Can you explain why we need an "Ambient" hack?



Performance Tip

Efficiency: To maximise GPU throughput, calculate the Normal Matrix $(M^{-1})^T$ on the CPU once per object rather than performing expensive matrix inversions inside the shader.

How to Approach This Lecture

A Layered Learning Strategy

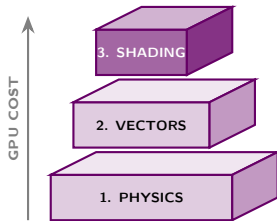
To master these concepts, treat the lecture as three distinct layers of a 3D "Reality Recipe."

1 Layer 1: The Physics (The "Why")

- Understand the **Blinn-Phong Components**: Ambient (Global), Diffuse (Matte), and Specular (Shiny).
- *Study Goal*: Can you explain why we need an "Ambient" hack?

2 Layer 2: The Vectors (The "How")

- Master the **Dot Product** ($N \cdot L$) for Diffuse and the **Halfway Vector** (H) for Specular.
- **CRITICAL**: Memorise the Normal Matrix $(M^{-1})^T$ requirement for non-uniform scaling.



Performance Tip

Efficiency: To maximise GPU throughput, calculate the Normal Matrix $(M^{-1})^T$ on the CPU once per object rather than performing expensive matrix inversions inside the shader.

How to Approach This Lecture

A Layered Learning Strategy

To master these concepts, treat the lecture as three distinct layers of a 3D "Reality Recipe."

1 Layer 1: The Physics (The "Why")

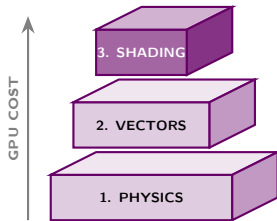
- Understand the **Blinn-Phong Components**: Ambient (Global), Diffuse (Matte), and Specular (Shiny).
- Study Goal*: Can you explain why we need an "Ambient" hack?

2 Layer 2: The Vectors (The "How")

- Master the **Dot Product** ($N \cdot L$) for Diffuse and the **Halfway Vector** (H) for Specular.
- CRITICAL**: Memorise the Normal Matrix ($M^{-1})^T$ requirement for non-uniform scaling.

3 Layer 3: The Sampling (The "Where")

- Compare **Flat**, **Gouraud**, and **Phong Shading**.
- Study Goal*: Understand why Phong Shading (per-fragment) is the modern standard for highlights.



Performance Tip

Efficiency: To maximise GPU throughput, calculate the Normal Matrix ($M^{-1})^T$ on the CPU once per object rather than performing expensive matrix inversions inside the shader.

Appendix: Calculating Surface Normals

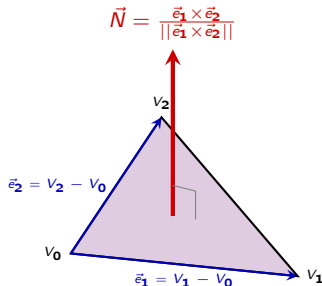
Per-Face vs. Per-Vertex Normals

1. Per-Face (Flat) Normals

- Single normal for the entire polygon.
- Calculated via the **Cross Product** of two edges ($V_1 \rightarrow V_0$ and $V_2 \rightarrow V_0$):

$$\vec{N}_{face} = \frac{(V_1 - V_0) \times (V_2 - V_0)}{\| (V_1 - V_0) \times (V_2 - V_0) \|}$$

- Results in a **faceted** look.

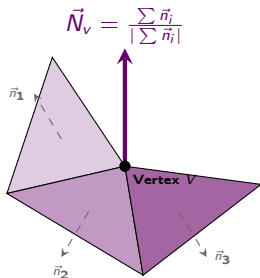


2. Per-Vertex (Smooth) Normals

- Unique normal for each vertex.
- Calculated by **averaging** the normals of all faces sharing that vertex:

$$\vec{N}_v = \frac{\sum \vec{N}_{face_i}}{\|\sum \vec{N}_{face_i}\|}$$

- Enables **smooth shading** (e.g., Gouraud/Phong).

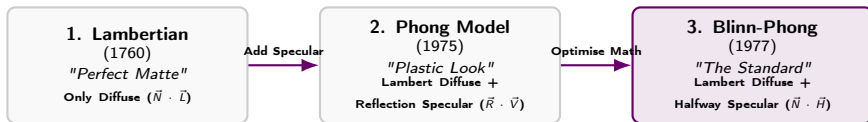


Appendix: Relationship Between Models

Evolution of the Standard Pipeline

Students often ask: *"Do I use Lambert OR Blinn-Phong?"*

The answer is: **Blinn-Phong INCLUDES Lambert.**



The "Classic" Graphics Pipeline

The Composition

Blinn-Phong is not a separate entity. It is a composite model:

- **Diffuse Part:** Uses Lambert's Law.
- **Specular Part:** Uses Blinn's approximation.

What comes next? (PBR)

Modern engines (Unreal/Unity) use **PBR (Physically Based Rendering)**.

PBR still uses the Halfway Vector ($\vec{H}$), but replaces simple dot products with complex physics (Fresnel, Geometry, Distribution) to conserve energy.

Appendix: Ambient vs. Diffuse vs. Specular

From Concept to Code

Component	Conceptual Role	Mathematical Dependency	High-Level (Three.js)
Ambient	Base "Global Glow". Simulates indirect light scattering. Prevents pitch-black shadows.	Constant. Independent of direction. $I = K_a$	'AmbientLight' (Added to Scene)
Diffuse	Matte Surface Colour. Reveals 3D form/depth.	Light Angle. Depends on Light ($\vec{L}$) vs. Normal ($\vec{N}$). $I = K_d \max(0, \vec{N} \cdot \vec{L})$	'color' property in 'MeshPhongMaterial'
Specular	Shiny Highlight. Simulates surface smoothness/gloss.	View Angle. Depends on Reflection ($\vec{R}$) or Halfway ($\vec{H}$) vs. View ($\vec{V}$). $I = K_s (\vec{N} \cdot \vec{H})^n$	'specular' (colour) & 'shininess' (intensity) in 'MeshPhongMaterial'

Summary

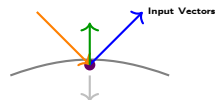
- **Ambient** is flat and directionless.
- **Diffuse** depends on where the light is coming from.
- **Specular** depends on where *you* (the viewer) are standing.

Appendix: Conceptual Distinction

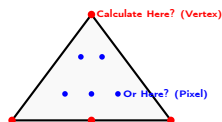
The "What" vs. The "How"

Feature	Lighting Model	Shading Model
Definition	The physics formula for calculating colour at a single point.	The strategy for applying that formula across a polygon.
Key Question	"What colour is this specific dot?"	"Where do we calculate the dots?"
Examples	Lambert, Phong, Blinn-Phong, PBR.	Flat, Gouraud, Phong.
Input	Light Vector ($\vec{L}$), Normal ($\vec{N}$), View ($\vec{V}$).	Vertex Normals, Interpolation Rules.

1. LIGHTING (The Formula)



2. SHADING (The Strategy)



Analogy

Lighting is the recipe (ingredients + method). **Shading** is deciding whether to cook the whole meal (per-pixel) or just show a picture of it (per-vertex).

Appendix: Core Three.js Materials

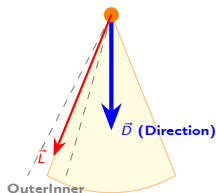
Material	Definition	Key Feature	Key API Properties	Usage	Things to Pay Attention To
MeshBasic-Material	A material for drawing geometries in a simple, unlit way.	Ignores all scene lighting.	color map wireframe	Wireframes, UI elements, flat backgrounds.	Will look completely 2D/flat. Needs baked-in shadows on the texture to show any depth.
MeshLambert-Material	A material for non-shiny (Lambertian) surfaces.	Calculates diffuse lighting only; no specular highlights.	color emissive map	Matt surfaces (e.g., paper, untreated wood, clay).	Better performance than Phong, but cannot simulate glossy reflections or shininess.
MeshPhong-Material	A material for shiny surfaces using the Blinn-Phong model.	Simulates glossy specular highlights.	color specular shininess	Shiny plastics, varnished wood, coated surfaces.	More computationally expensive than Lambert. Specular colour often needs manual tweaking to look natural.
MeshStandard-Material	A physically based rendering (PBR) material.	Uses real-world physics (roughness/metalness workflow).	color roughness metalness	Modern, highly realistic 3D scenes and assets.	Heaviest performance cost. Strongly relies on good lighting and environment maps (HDRI) for optimal behaviour.

Unlike a Point Light which radiates uniformly (360°), a **Spotlight** is constrained to a cone defined by a **Direction Vector** ($\vec{D}$) and **Cut-off Angles**.

1. The Angular Alignment

- Let $\vec{L}$ be the vector from the light to the fragment.
- The alignment is calculated via the **Dot Product**:

$$\cos(\theta) = \text{unit}(-\vec{L}) \cdot \text{unit}(\vec{D})$$



Point Light: $f(\text{dist})$
Spotlight: $f(\text{dist}) \times f(\text{angle})$

2. The Penumbra (Soft Edges)

- To avoid hard aliased edges, we define an **Inner** and **Outer** cone.
- Intensity Formula (I):**

$$I = \text{clamp} \left(\frac{\cos \theta - \cos \phi_{\text{outer}}}{\cos \phi_{\text{inner}} - \cos \phi_{\text{outer}}}, 0, 1 \right)$$

- Result: Full brightness inside the inner cone, fading to zero at the outer edge.

Appendix: Shader for Spotlight Logic

The following GLSL logic integrates the angular constraint into the standard lighting equation. Note that cosines are pre-calculated on the CPU to ensure the fragment shader is **optimised**.

```
// UNIFORMS (Passed from CPU/Three.js)
uniform vec3 uSpotLightPos;    // Light position in world space
uniform vec3 uSpotLightDir;    // Normalised direction vector (forward)
uniform float uInnerCos;       // cos(inner_angle)
uniform float uOuterCos;       // cos(outer_angle)

void main() {
    vec3 lightVec = uSpotLightPos - vWorldPos;
    vec3 L = normalize(lightVec);

    // 1. Calculate cosine of the angle between light direction and fragment
    // We use -L because L points from surface to light
    float cosTheta = dot(normalize(-L), normalize(uSpotLightDir));

    // 2. Calculate smooth intensity (The Penumbra)
    float epsilon = uInnerCos - uOuterCos;
    float spotIntensity = clamp((cosTheta - uOuterCos) / epsilon, 0.0, 1.0);

    // 3. Distance Attenuation (Standard Point Light Component)
    float d = length(lightVec);
    float attenuation = 1.0 / (1.0 + 0.01 * d + 0.002 * (d * d));

    // 4. Combine: Ambient + (Diffuse * Spotlight Constraint * Attenuation)
    vec3 diffuse = max(dot(vNormal, L), 0.0) * uLightColour;
    vec3 finalColour = uAmbient + (diffuse * spotIntensity * attenuation);

    gl_FragColor = vec4(finalColour, 1.0);
}
```

Key Implementation Difference

While Point Lights only require **Position**, a Spotlight requires **Position + Direction**. The 'spotIntensity' acts as a mask that "carves" the cone out of a standard point light's radial emission.

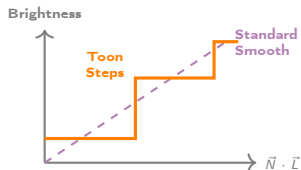
Appendix: Cartoon (Toon) Shading

Non-Photorealistic Rendering (NPR)

To achieve a "hand-drawn" or "cel-shaded" look, we deliberately discard physical realism. Instead of allowing the light intensity to fade smoothly, we **quantise** it into discrete bands.

The Logic: Discretisation

- Standard Diffuse ($\vec{N} \cdot \vec{L}$) creates a continuous gradient (0.0 $\rightarrow$ 1.0).
- Toon Shading forces this value into specific "buckets" (e.g., Shadow, Mid-tone, Bright).

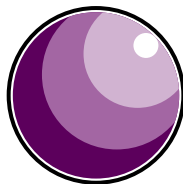


GLSL Implementation

```
// 1. Calculate standard intensity (0.0 to 1.0)
vec3 N = normalize(v_normal);
vec3 L = normalize(u_lightDir);
float intensity = max(0.0, dot(N, L));

// 2. Quantise into 3 bands (The "Step" Logic)
if (intensity > 0.95)
    intensity = 1.0; // Sharp Highlight
else if (intensity > 0.5)
    intensity = 0.7; // Mid-tone
else
    intensity = 0.2; // Shadow Base

// 3. Output flat colour
gl_FragColor = vec4(u_color * intensity, 1.0);
```



Expected Result

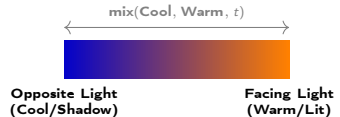
Appendix: Gooch Shading

Technical Illustration (Warm-to-Cool Shift)

Standard lighting makes shadows black, which hides geometric detail. **Gooch Shading** (used in engineering) solves this by mapping light intensity to **Colour Temperature** rather than brightness.

The Logic: Interpolation

- Instead of fading to black, we shift between two distinct colours.
- **Sunlit surfaces** → Warm (Yellow/Orange).
- **Shadowed surfaces** → Cool (Blue/Purple).
- Result: Detail remains visible even in "dark" areas.



GLSL Implementation

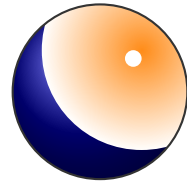
```
// 1. Calculate Dot Product (-1.0 to 1.0)
vec3 N = normalize(v_normal);
vec3 L = normalize(u_lightDir);
float NdotL = dot(N, L);

// 2. Remap [-1, 1] range to [0, 1]
// 0.0 = Shadow side, 1.0 = Lit side
float t = (NdotL + 1.0) * 0.5;

// 3. Interpolate (Mix) colors
vec3 cool = vec3(0.0, 0.0, 0.5); // Deep Blue
vec3 warm = vec3(1.0, 0.6, 0.0); // Orange

// When t=0, we get 100% Cool. When t=1, we get 100% Warm.
// FUNCTION: mix(x, y, a) = x*(1-a) + y*a
vec3 final = mix(cool, warm, t);

gl_FragColor = vec4(final, 1.0);
```



Result: No Black Shadows

Appendix: Pencil Sketch Shading

Non-Photorealistic Rendering (Post-Processing)

Unlike physical light, a pencil sketch relies on **outlines** and **cross-hatching** to convey form. This is typically done as a 2D post-processing effect over the rendered image.

The Logic: Edges & Hatching

- **Edge Detection:** We sample adjacent pixels (texels). If their brightness (luminance) differs sharply, it is an edge.
- **Cross-Hatching:** We use the screen coordinates (vUv) to mathematically generate repeating diagonal lines, applying them progressively to darker areas.

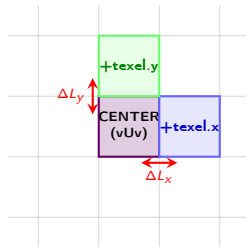
GLSL Implementation

```
// 1. Edge Detection (Compare neighbors)
float lum = luminance(texture2D(tDiffuse, vUv).rgb);
float lum1 = luminance(texture2D(tDiffuse, vUv + vec2(texel.x, 0.0)).rgb);
// Large difference = dark edge
float edge = length(vec2(lum - lum1)) * 6.0;
float sketch = 1.0 - edge; // Base paper (white)

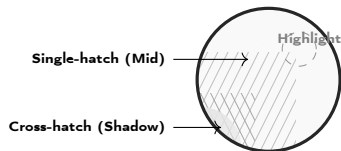
// 2. Procedural Hatching (Math based on UVs)
float hatch1 = fract((vUv.x + vUv.y) * 200.0);
float hatch2 = fract((vUv.x - vUv.y) * 200.0);

// Apply hatching to shadow areas
if (lum < 0.5 && hatch1 > 0.5) sketch -= 0.15; // Mid-tone
if (lum < 0.3 && hatch2 > 0.5) sketch -= 0.25; // Dark shadow

// Clamp to prevent pure black/white
gl_FragColor = vec4(vec3(clamp(sketch, 0.1, 0.95)), 1.0);
```



Edge Detection Logic
If $\sqrt{\Delta L_x^2 + \Delta L_y^2} > \text{threshold}$
Result = Dark Outline



Final Fragment Output

Appendix: Chinese Water Ink Shading

Organic Noise & Calligraphic Rendering

A Chinese water ink (Shuimo) shader simulates organic ink spreading on highly absorbent rice paper. This relies heavily on **Fractal Brownian Motion (FBM)** and **Noise** to create procedural distortion, varying brush pressure, and dry brush strokes.

The Logic: Organic Distortion

- **Brush Pressure:** Instead of a fixed 1-pixel edge detection, noise dynamically expands the Sobel sampling distance, creating thick and thin stroke variations.
- **Feibai (Dry Brush):** High-frequency noise randomly "erases" parts of the outline, mimicking a brush running out of ink.
- **Ink Bleed (Mogu):** UV coordinates are warped using FBM before sampling the base image, simulating ink bleeding into wet paper.

GLSL Implementation (Key Steps)

```
// 1. Variable Edge Thickness (Simulated Pressure)
float brushPressure = noise(uv * 12.0);
float stepDist = 1.0 + brushPressure * 3.0; // Distort radius
// ... Sobel sampling using stepDist ...
float edge = sqrt(sx*sx + sy*sy);

// 2. "Feibai" (Dry Brush / Random Fading)
float bristles = noise(uv * 400.0); // High-freq marks
float fadeSpots = fbm(uv * 20.0); // Organic gaps
float lineBreak = smoothstep(0.2, 0.8,
    bristles * 0.4 + fadeSpots + brushPressure * 0.4);

// Cut the solid edge using the noise breaks
float outline = smoothstep(0.05, 0.25, edge) * lineBreak;

// 3. Wash & Paper Composite
float paperGrain = noise(uv * 800.0) * 0.04;
vec3 paperColor = vec3(0.96, 0.94, 0.91) - paperGrain;

// mix base wash with aggressive dark ink strokes
vec3 finalColor = mix(paperColor * washTone, inkColor, outline);
gl_FragColor = vec4(finalColor, 1.0);
```

1. Variable Edge Spread



Noise expands Sobel sampling
= varying stroke thickness

2. Feibai (Dry Brush)



High-freq bristle noise
erases parts of the solid edge

3. Ink Bleed (Warping)



FBM distorts UV coordinates
creating organic capillary bleed

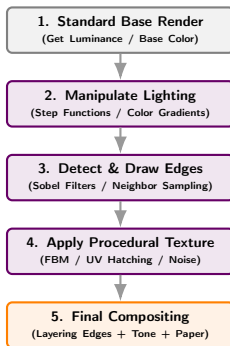
Appendix: Crafting Stylistic Shaders

Summary: Core Techniques for Non-Photorealistic Rendering

Creating a "hand-drawn" or stylistic look requires breaking the rules of physical lighting. Across Toon, Gooch, Pencil, and Ink shaders, we rely on a shared toolbox of four major high-level techniques:

- 1 Lighting Manipulation (Quantisation & Mapping):** Abandoning continuous, linear light falloff. Using `if/else` step functions to quantise light into discrete bands (Toon), or mapping intensity to colour temperatures via `mix()` (Gooch).
- 2 Edge Detection (Outlining):** Finding the boundaries of geometry or shadows. Sampling adjacent texels (Sobel filtering) to find sharp differences in luminance, drawing lines where depth or light changes abruptly.
- 3 Procedural Math (Screen-Space Texturing):** Generating drawing mediums mathematically. Using screen coordinates (`vUv`) and math functions (`sin`, `fract`) to generate cross-hatching, or using Fractal Brownian Motion (FBM) for organic ink bleeds and paper grain.
- 4 Compositing (Layering):** Building the final illusion. Multiplying base colors by texture masks, applying dark stroke outlines over soft washes, and overlaying paper grain as a final step.

The Stylistic Shading Pipeline



Executed
per-pixel in
Fragment
Shader